Smart Parking Lot Management System - Business Requirement & Abstract (Artistic)

Business Requirement Document

Project Title:

Smart Parking Lot Management System

Abstract

The Smart Parking Lot Management System aims to revolutionize urban parking by providing a scalable, flexible, and extensible platform for managing multi-floor parking lots with multiple entry and exit gates. The system supports diverse vehicle types (Car, Bike, Truck, EVs), ensures optimal spot allocation, and integrates additional customer services (e.g., car wash, EV charging). It features dynamic pricing strategies, seamless payment integration, and real-time capacity monitoring. The backend is architected using SOLID principles and leverages creational, structural, and behavioral design patterns to ensure maintainability, extensibility, and adaptability for future requirements.

Scope

Core Features

- **Multi-Floor Parking Management:**

Supports multiple floors, each with distinct capacities for various vehicle types.

- **Entry/Exit Gate Operations:**

Ticket generation at entry gates, invoice generation at exit gates, and robust validation to prevent misuse.

- **Vehicle Spot Allocation:**

Assigns nearest available spot based on vehicle type and floor availability, with flexible strategies for spot assignment (open for future changes).

- **Dynamic Pricing Models:**

Supports both slab-based and fixed pricing, configurable for weekdays/weekends and vehicle types. Pricing logic is encapsulated using strategy patterns for easy extension.

- **Additional Services:**

Offers optional services (car wash, EV charging, detailing) with vehicle-type validation and fixed pricing, seamlessly integrated into ticketing and invoicing.

- **Capacity Monitoring API:**

Provides real-time capacity information at parking lot, floor, and vehicle-type granularity.

- **Payment Integration:**

Integrates with Razorpay for secure, flexible payment processing.

Backend Strategy

- **SOLID Principles:**

All classes and interfaces are designed for single responsibility, open/closed for extension, and dependency inversion. This ensures each module is independently testable and modifiable.

- **Creational Design Patterns:**

Extensive use of Factory and Builder patterns for creating tickets, invoices, vehicles, and services, allowing easy addition of new vehicle types or services.

- **Structural Patterns:**

Adapter and Composite patterns are used to unify APIs for capacity reporting and payment integration, enabling seamless expansion and integration with third-party systems.

- **Behavioral Patterns:**

Strategy pattern for pricing and spot allocation logic, Observer pattern for real-time capacity updates, and Command pattern for gate operations.

- **Extensibility & Flexibility:**

The architecture is intentionally open for major extensions—new pricing models, vehicle types, services, or integrations can be added with minimal changes to core logic. All business rules (e.g., spot assignment, pricing, service eligibility) are encapsulated in strategy classes, making the system highly adaptable.

Future-Proofing

- **Open for Change:**

Spot assignment, pricing, and service logic are abstracted and injected via interfaces, allowing rapid adaptation to new business requirements.

- **Modular Service Integration:**

Additional services (e.g., valet, loyalty programs) can be plugged in without refactoring existing code.

- **In-Memory Database:**

Initial implementation uses an in-memory database for rapid prototyping and testing, with clear separation for future migration to persistent storage.

Creative Features

- **Real-Time Analytics:**

Live dashboard for operators to monitor occupancy, revenue, and service usage.

- **Customer Personalization:**

Ability to offer targeted promotions or bundled services based on vehicle history.

- **API-First Design:**

All features are exposed via RESTful APIs, enabling integration with mobile apps, kiosks, and third-party platforms.

Summary

This project delivers a robust, extensible, and future-ready parking lot management system, leveraging advanced backend design principles and patterns. Its modular architecture ensures that new features, pricing models, and integrations can be added with ease, making it ideal for evolving urban mobility needs.
